Redes Neuronales III: optimización, regularización y evaluación de modelos de deep learning sobre MNIST con TensorFlow, Keras y PyTorch

Yohann Javier Nikolay Salazar Cardenas

Docente

Nidia Stella Garcia Roa

Universidad de Cundinamarca

Especialización en Inteligencia Artificial

604013/DEEP LEARNING -CONCEPTOS/FU/CAD2202023205/EIAIIPA2026

2026

Tabla de Contenido

Introducción 3
Fundamentos Conceptuales 4
Parámetros e Hiperparámetros 4
Train, Dev y Test 4
Optimizadores 4
Regularización y Early Stopping 5
Bias, Variance y Métricas de Clasificación 5
Metodología 5
Resultados 6
Datos, Preprocesamiento y Modelo Baseline 6
Comparación de Optimizadores y Learning Rate 7
Regularización y Control del Overfitting 9
Evaluación Final sobre Test 10
Réplica del Entrenamiento con PyTorch 12
Conclusiones 13
Referencias Bibliográficas 15
Apéndice A: Código del Notebook 01 (Datos y Baseline) 16
Apéndice B: Código del Notebook 02 (Optimización) 20
Apéndice C: Código del Notebook 03 (Regularización) 24
Apéndice D: Código del Notebook 04 (Métricas) 29
Apéndice E: Código del Notebook 05 (PyTorch) 33

Introducción

En la actividad previa se estableció que una red neuronal aprende cuando el error de sus predicciones se utiliza para modificar sus parámetros mediante backpropagation y descenso de gradiente (Trask, 2019), de modo que el propósito de esta actividad es someter ese aprendizaje a control experimental, es decir, decidir contra qué conjunto de datos se comparan las configuraciones, cuándo detener el entrenamiento, cómo evitar que el modelo memorice y con qué métricas afirmar que generaliza, pues sin ese control cualquier cifra de desempeño resultaría anecdótica.

Todas las prácticas comparten un mismo problema, a saber, la clasificación de dígitos manuscritos de MNIST (LeCun et al., 2010) con redes densas sin convoluciones, y varían un único factor por experimento, sea el optimizador, el learning rate o la regularización, ya que esa disciplina de experimentos controlados es la que permite atribuir cada diferencia observada a la decisión estudiada y no al azar de los datos; el documento sigue, por tanto, la progresión de la guía del curso: datos y línea base, optimización, regularización, evaluación con métricas por clase y réplica del entrenamiento en PyTorch (Stevens et al., 2020).

La tesis que organiza el trabajo puede enunciarse así: un modelo de deep learning no se valida con una cifra de accuracy, sino con un protocolo, y el protocolo empleado separa Train, Dev y Test con funciones distintas, compara configuraciones únicamente sobre Dev, reserva Test para una única evaluación final y desagrega el desempeño con Precision, Recall, F1 y matriz de confusión, de tal forma que el código completo y ejecutado se publica en el repositorio de la actividad (Salazar Cardenas, 2026).

Fundamentos Conceptuales

Parámetros e Hiperparámetros

Los parámetros, esto es, pesos y sesgos, los ajusta el optimizador dentro del ciclo de entrenamiento, mientras que los hiperparámetros, como el learning rate, el batch size, el número de épocas y neuronas o la fuerza de regularización, los fija quien experimenta y se deciden fuera del ciclo comparando configuraciones sobre Dev, de modo que confundir ambos niveles conduce a ajustar el modelo mirando métricas contaminadas.

Train, Dev y Test

Train actualiza los pesos, Dev compara configuraciones y monitorea la generalización durante el diseño, por lo que elegir optimizador o learning rate mirando Dev es legítimo, y Test se reserva para una única evaluación final, dado que utilizarlo para elegir optimizadores o hiperparámetros contamina la estimación de generalización y constituye un error metodológico conocido como data leakage; la partición estratificada, por su parte, conserva la proporción de clases en los tres conjuntos.

Optimizadores

El gradiente señala la dirección del paso y el optimizador determina su tamaño y trayectoria, de tal manera que SGD con momentum acumula parte de los pasos previos para conservar dirección, RMSprop normaliza el paso con el promedio reciente de gradientes al cuadrado y Adam combina momentum y adaptación por segundo momento (Gulli et al., 2023); ninguno de los tres es ganador universal, puesto que su desempeño relativo depende del problema, la arquitectura y el horizonte de épocas, razón por la cual se compara empíricamente con todo lo demás fijo.

Regularización y Early Stopping

La regularización limita la capacidad efectiva del modelo para que no memorice ruido, y así la penalización L2 añade a la pérdida el término lambda por la suma de pesos al cuadrado favoreciendo soluciones suaves, Dropout apaga aleatoriamente una fracción de unidades durante el entrenamiento obligando a distribuir la representación y evitando dependencias de neuronas individuales (Srivastava et al., 2014), y Early Stopping vigila la pérdida de validación deteniendo el entrenamiento cuando deja de mejorar durante un número patience de épocas para restaurar los pesos de la mejor época, lo que equivale a regularizar por tiempo de entrenamiento.

Bias, Variance y Métricas de Clasificación

Un modelo con bias alto rinde mal incluso en Train, en tanto que uno con variance alta brilla en Train y cae en Dev, por lo que el diagnóstico se hace observando la brecha Train-Dev en las curvas de pérdida y no con una sola cifra; para el desempeño final, además, el accuracy resulta insuficiente, pues Precision mide cuántas predicciones de una clase fueron correctas, Recall cuántos ejemplos reales de la clase se recuperaron y F1 es la media armónica de ambas, de modo que con diez clases se reporta el promedio macro y la matriz de confusión completa el diagnóstico mostrando qué pares de clases se confunden y convirtiendo el error agregado en información accionable (Gulli et al., 2023).

Metodología

Las cinco prácticas se desarrollaron como notebooks de Google Colaboratory ejecutados de extremo a extremo con semilla 42, y de las 60 000 imágenes de entrenamiento de MNIST se derivaron Train (48 000) y Dev (12 000) mediante partición estratificada, mientras que Test (10 000)

permaneció intocado hasta la evaluación final; los píxeles se normalizaron al intervalo entre cero y uno, se entrenaron dos arquitecturas, una línea base 784-128-10 y una red grande 784-512-256-10 para el experimento de regularización, esta última sobre una muestra estratificada de 12 000 imágenes, y la Tabla 1 resume el entorno.

Tabla 1

Entorno de Trabajo y Particiones de Datos

Componente	**Versión o valor**
Lenguaje	Python 3.11
Frameworks	TensorFlow 2.21.0 con Keras 3.15.1; PyTorch 2.14.1
Métricas	scikit-learn 1.9.1
Dataset	MNIST: 60 000 imágenes de entrenamiento y 10 000 de prueba
Particiones	Train 48 000, Dev 12 000, Test 10 000 (estratificadas, semilla 42)
Arquitecturas	784-128-10 (baseline) y 784-512-256-10 (regularización)
Dispositivo	CPU de dos núcleos con aproximadamente 1 GB de RAM

Nota. Los entrenamientos se escribieron como ciclos explícitos por lotes, matemáticamente idénticos a model.fit(), para mantener el consumo de memoria dentro del límite del entorno.

Resultados

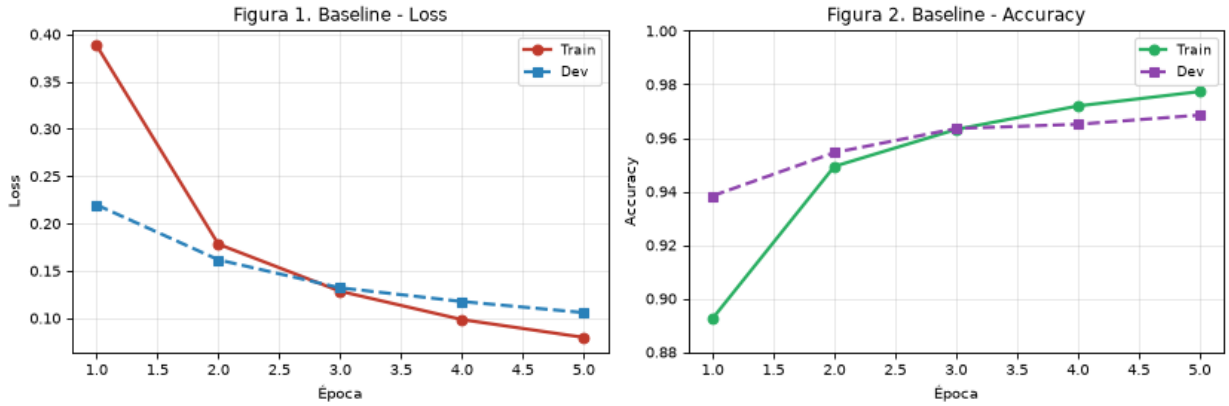
Datos, Preprocesamiento y Modelo Baseline

La partición estratificada produjo desviaciones máximas de 0.01 puntos porcentuales en la proporción de clases entre Train y Dev, con los diez dígitos presentes en los tres conjuntos, y el baseline de 101 770 parámetros, sin capas convolucionales, redujo su pérdida de Dev de 0.2196 en la primera época a 0.1058 en la quinta, con exactitud de Dev de 0.9685; la Figura 1 muestra que las curvas de Train y Dev avanzan juntas, de modo que con esta capacidad y horizonte no aparece divergencia de overfitting, y

este modelo quedó fijado como referencia comparativa de los experimentos siguientes.

Figura 1

Curvas de pérdida y exactitud del modelo baseline en Train y Dev



Nota. Elaboración propia a partir de la ejecución del Notebook 01 (Salazar Cardenas, 2026).

Comparación de Optimizadores y Learning Rate

Con arquitectura, datos, épocas y semilla idénticos, RMSprop obtuvo el mejor Dev (0.9723), Adam quedó en 0.9698 y SGD con momentum en 0.9627, como resumen la Tabla 2 y la Figura 2, y la exploración del learning rate de Adam mostró el comportamiento multiplicativo del hiperparámetro, pues con 1e-4 el modelo subajusta (Dev 0.9213), con 1e-3 alcanza su mejor punto (0.9675) y con 1e-2 oscila (0.9595); la conclusión correcta, no obstante, no es que un optimizador sea universalmente superior, sino que bajo las condiciones de este experimento las diferencias entre familias fueron menores que las diferencias entre learning rates.

Tabla 2

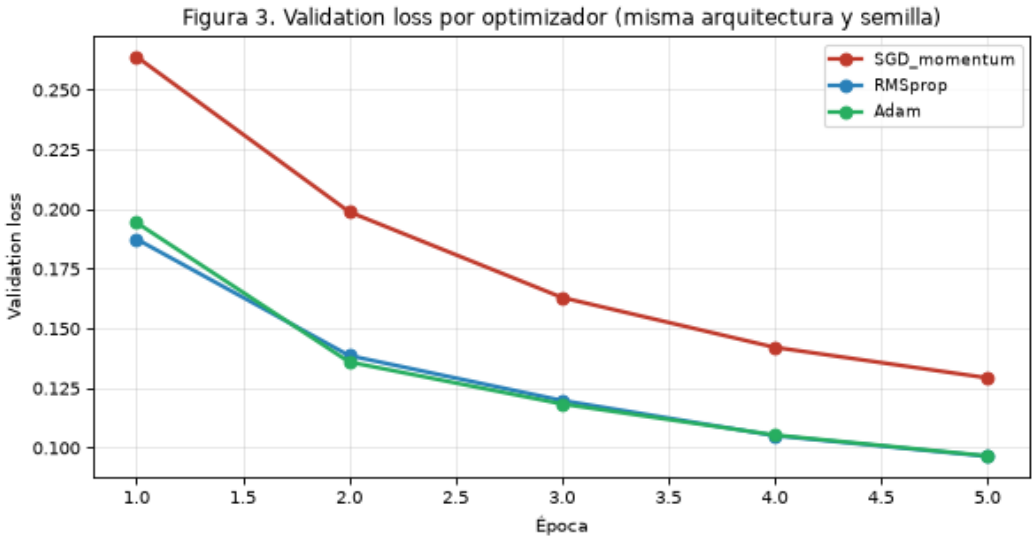
Comparación de Optimizadores en la Época 5

Optimizador	Train accuracy	Dev accuracy	Dev loss
SGD con momentum (lr 0.01)	0.9672	0.9627	0.1293
RMSprop (lr 0.001)	0.9800	0.9723	0.0964
Adam (lr 0.001)	0.9811	0.9698	0.0966

Nota. Misma arquitectura 784-128-10, mismos datos y semilla 42, de modo que las decisiones se tomaron sobre Dev.

Figura 2

Validation loss por optimizador con idéntica arquitectura y semilla



Nota. Elaboración propia a partir de la ejecución del Notebook 02.

Tabla 3

Efecto del Learning Rate de Adam con Tres Épocas

Learning rate	Train accuracy	Dev accuracy	Dev loss
0.0001	0.9170	0.9213	0.2851
0.0010	0.9686	0.9675	0.1099
0.0100	0.9673	0.9595	0.1583

Nota. El efecto del learning rate es multiplicativo: cada fila multiplica el paso por diez respecto de la anterior.

Regularización y Control del Overfitting

La red grande de 535 818 parámetros entrenada sobre 12 000 imágenes sin regularización mostró la firma clásica del overfitting, ya que su pérdida de Train colapsó a 0.0002 en la época 15 mientras la de Dev se estancó alrededor de 0.14 (Figura 3), en tanto que el modelo regularizado con L2, Dropout y Early Stopping no pudo memorizar: detuvo el entrenamiento en la época 9 al cumplirse el patience de dos épocas sin mejora y, con los pesos restaurados a la mejor época, logró mejor Dev que el modelo sin regularización, esto es, loss de 0.1287 frente a 0.1345 y accuracy de 0.9641 frente a 0.9609, como resume la Tabla 4.

Tabla 4

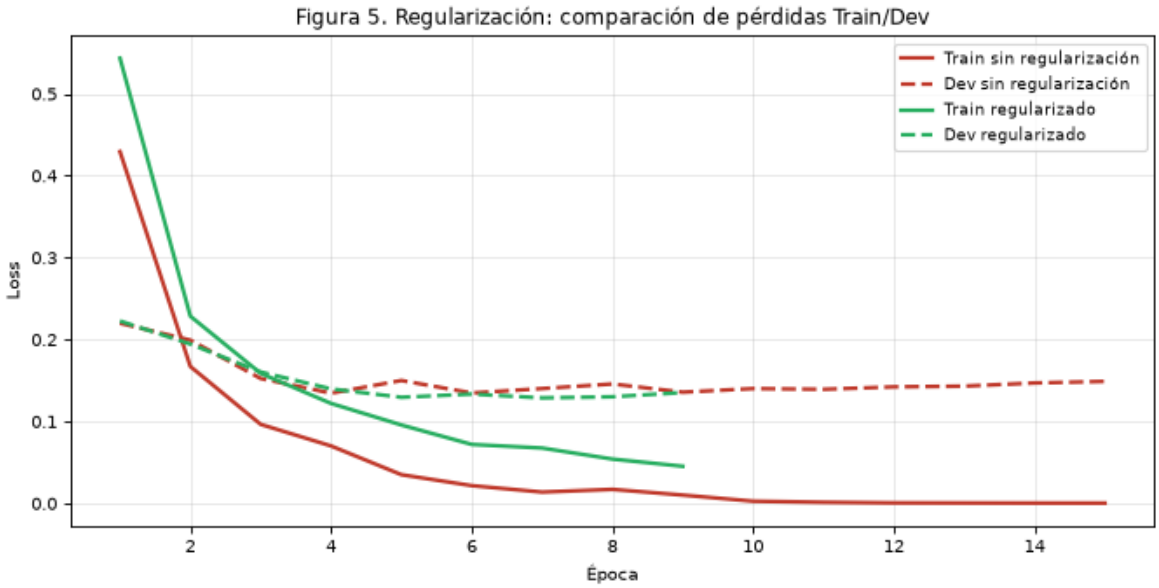
Comparación de Modelos con Pesos Restaurados a la Mejor Época

Modelo	Épocas	Train accuracy	Dev loss	Dev accuracy	Brecha Train-Dev
Sin regularización	15	0.9912	0.1345	0.9609	0.0302
L2 + Dropout + Early Stopping	9	0.9948	0.1287	0.9641	0.0308

Nota. Muestra de 12 000 imágenes y arquitectura 784-512-256-10 en ambos casos, de modo que el modelo regularizado alcanzó mejor Dev con seis épocas menos.

Figura 3

Pérdidas Train y Dev sin regularización y con regularización



Nota. Elaboración propia a partir de la ejecución del Notebook 03, donde la separación entre Train y Dev del modelo sin regularización constituye la firma del overfitting.

La lectura metodológica es la más relevante, dado que con los pesos restaurados ambos modelos alcanzan exactitudes de Train similares, de modo que únicamente Dev permite distinguir cuál generaliza mejor, pues evaluar mirando Train habría conducido a una elección indiferente o equivocada.

Evaluación Final sobre Test

El modelo regularizado se evaluó por primera y única vez sobre Test, donde obtuvo accuracy de 0.9644 con 356 errores en 10 000 imágenes, Precision macro de 0.9643, Recall macro de 0.9640 y F1 macro de 0.9641 (Tabla 5), y el reporte por clase identificó al dígito 8 como la clase más débil, con F1 de 0.9489, mientras la matriz de confusión localizó la confusión dominante en el par 4 hacia 9, con 20 casos, seguida de 9 hacia 8,

9 hacia 4 y 5 hacia 8; la Figura 4 presenta la matriz completa y la Figura 5 ejemplos reales del par confundido, que resultan ser trazos efectivamente ambiguos, lo cual contextualiza el límite del modelo sin convoluciones.

Tabla 5

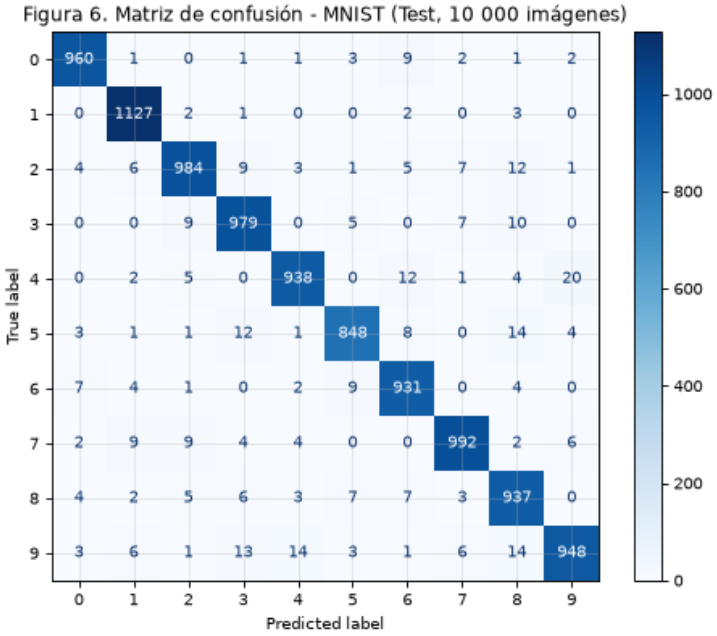
Métricas Finales del Modelo Regularizado sobre Test

Métrica	**Valor**	**Lectura**
Accuracy	0.9644	356 errores de 10 000 imágenes
Precision macro	0.9643	Confianza calibrada entre clases
Recall macro	0.9640	Cobertura pareja de los diez dígitos
F1 macro	0.9641	Equilibrio armónico entre Precision y Recall
Clase más débil	Dígito 8 (F1 0.9489)	Trazos cerrados confundibles con 3, 9 y 5
Confusión dominante	4 hacia 9 (20 casos)	Par morfológicamente similar

Nota. Test se utilizó una sola vez; ningún ajuste posterior se basó en estos números.

Figura 4

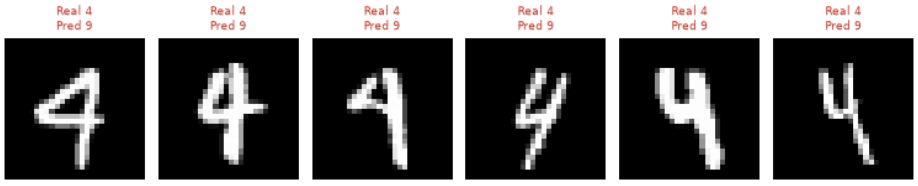
Matriz de confusión del modelo final sobre las 10 000 imágenes de Test



Nota. Elaboración propia a partir de la ejecución del Notebook 04.

Figura 5

Ejemplos reales de la confusión más frecuente (4 clasificado como 9)



Nota. Elaboración propia a partir de la ejecución del Notebook 04, y la confianza del modelo en el error analizado fue de 0.6726.

Réplica del Entrenamiento con PyTorch

La misma lógica se replicó en PyTorch con ciclo explícito de cinco pasos, a saber, limpiar gradientes, propagar hacia adelante, calcular la pérdida, propagar hacia atrás y actualizar, y con tres épocas el modelo alcanzó Dev accuracy de 0.9690 y Test accuracy de 0.9704 (Tabla 6 y Figura

6), de modo que la diferencia respecto de Keras se explica por arquitectura y muestras y no por el framework, pues ambos implementan el mismo ciclo matemático y el valor de PyTorch consiste en hacer visible lo que Keras encapsula.

Tabla 6

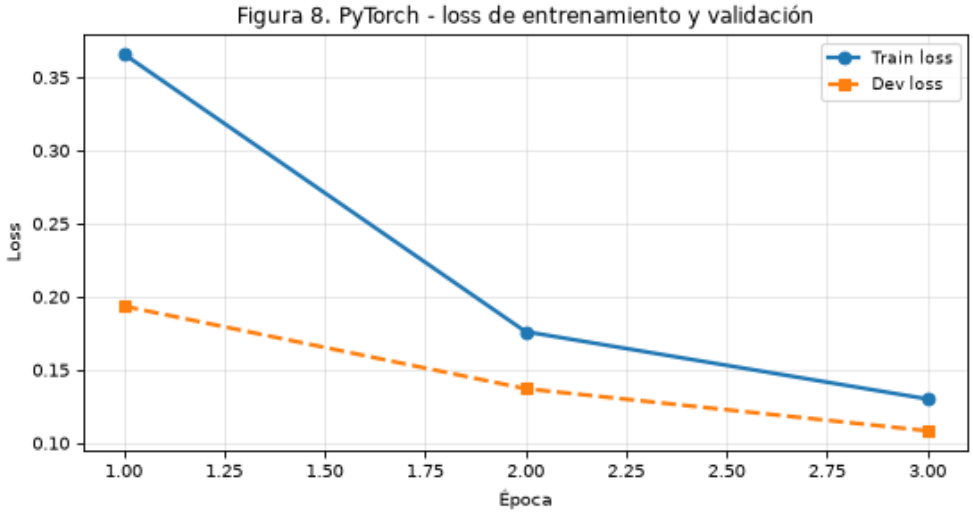
Entrenamiento del Modelo PyTorch por Época

Época	Train loss	Train accuracy	Dev loss	Dev accuracy
1	0.3659	0.8971	0.1935	0.9468
2	0.1761	0.9489	0.1371	0.9605
3	0.1300	0.9619	0.1083	0.9690

Nota. Arquitectura 784-256-10 con Dropout 0.30 y weight_decay 1e-4, con una exactitud final en Test de 0.9704.

Figura 6

Loss de entrenamiento y validación del modelo implementado en PyTorch



Nota. Elaboración propia a partir de la ejecución del Notebook 05.

Conclusiones

La separación de Train, Dev y Test con funciones distintas no es un formalismo, sino la condición que permite atribuir decisiones y estimar

generalización, por lo que Test se utilizó una sola vez, al final, y ningún ajuste posterior se basó en sus números.

El learning rate demostró ser el hiperparámetro más sensible del estudio, pues su efecto multiplicativo dominó sobre la elección de la familia del optimizador en horizontes cortos de entrenamiento, y así RMSprop y Adam empataron técnicamente en Dev mientras SGD quedó por detrás en cinco épocas, resultado condicionado al horizonte y a la arquitectura y no una jerarquía universal.

El overfitting se diagnosticó en la divergencia de las curvas Train-Dev y se controló combinando L2, Dropout y Early Stopping, de tal forma que el modelo regularizado mejoró Dev con nueve épocas frente a quince, lo que evidencia que la parada temprana es también un ahorro computacional.

El accuracy sin desglose resulta insuficiente, dado que Precision, Recall, F1 macro, reporte por clase y matriz de confusión localizaron el error residual en el dígito 8 y en el par 4 hacia 9, información directamente accionable para futuras mejoras, y la confianza de la predicción, notablemente menor en los errores, ofrece además una señal utilizable para umbrales de rechazo y revisión humana.

PyTorch y Keras implementan el mismo ciclo matemático de aprendizaje, por lo que dominar ambos niveles de explicitud permite elegir la herramienta según el problema sin reaprender la teoría, y finalmente la reproducibilidad, esto es, semillas fijas, versiones documentadas, salidas conservadas y código publicado, es lo que convierte estos resultados en evidencia verificable por terceros (Salazar Cardenas, 2026).

Referencias Bibliográficas

Google. (2026). Google Colaboratory [Entorno de notebooks en la nube]. Google Research. https://colab.research.google.com/

Gulli, A., Kapoor, A., Pal, S., & Ciraci, S. (2023). Deep learning with TensorFlow and Keras (3.ª ed.). Packt Publishing. https://download.packt.com/free-ebook/9781803232911

LeCun, Y., Cortes, C., & Burges, C. J. C. (2010). MNIST handwritten digit database. AT&T Labs Research. http://yann.lecun.com/exdb/mnist/

Salazar Cardenas, Y. J. N. (2026). Redes Neuronales III [Código fuente]. GitHub. https://github.com/Yohann0011/Redes-Neuronales-III

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), 1929-1958.

Stevens, E., Antiga, L., & Viehmann, T. (2020). Deep learning with PyTorch. Manning Publications. https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf

Trask, A. (2019). Grokking deep learning. Manning Publications. https://edu.anarcho-copy.org/Algorithm/grokking-deep-learning.pdf

Universidad de Cundinamarca. (2026). Guía de trabajo y laboratorio: Redes Neuronales III [Material del curso Deep Learning - Conceptos]. Especialización en Inteligencia Artificial.

Apéndice A

Código Fuente del Notebook 01: Datos, Preprocesamiento y Baseline

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import pandas as pd
import random, gc

random.seed(42)
tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
print("Backend    :", keras.backend.backend())

from sklearn.model_selection import train_test_split

SEED = 42
np.random.seed(SEED)

(x_train_full, y_train_full), (x_test_raw, y_test) = keras.datasets.mnist.load_data()

# División estratificada Train / Dev (80 / 20). Test queda intocable hasta el final.
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_full,
)

# Test se normaliza completo (31 MB); Train y Dev se normalizan por lotes para
# mantener el consumo de memoria del kernel bajo control.
x_test = x_test_raw.astype("float32") / 255.0


def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n, 28, 28) en float32 (n, 28, 28) dentro de [0, 1].

    El aplanado a 784 características lo realiza la capa Flatten del modelo.
    """
    return imagenes.astype("float32") / 255.0
```

```
print("Train:", x_train.shape, "| Dev:", x_dev.shape, "| Test:", x_test.shape)

print()
print("Distribución de clases (conteo por dígito):")
print("Train completo:", np.bincount(y_train_full))
print("Train  :", np.bincount(y_train))
print("Dev    :", np.bincount(y_dev))
print("Test   :", np.bincount(y_test))

def distribucion_clases(y, nombre):
    """Tabla con conteo y porcentaje de cada dígito en un conjunto."""
    conteos = np.bincount(y, minlength=10)
    porcentajes = conteos / len(y) * 100
    tabla = pd.DataFrame({
        "digito": np.arange(10),
        "cantidad": conteos,
        "porcentaje": np.round(porcentajes, 2),
    })
    print("\n", nombre)
    display(tabla)
    print(tabla.to_string(index=False))
    return tabla


dist_train = distribucion_clases(y_train, "TRAIN")
dist_dev = distribucion_clases(y_dev, "DEV")
dist_test = distribucion_clases(y_test, "TEST")

print()
print("¿Los diez dígitos están presentes en los tres conjuntos?",
      bool((dist_train["cantidad"] > 0).all() and (dist_dev["cantidad"] > 0).all()
           and (dist_test["cantidad"] > 0).all()))
print("Desviación máxima de porcentaje entre Train y Dev:",
      round(float(np.abs(dist_train["porcentaje"] - dist_dev["porcentaje"]).max()), 2), "puntos")

lote_crudo = x_train[:5]
lote_norm = normalizar_lote(lote_crudo)

print("Sin normalizar -> mínimo:", lote_crudo.min(), "máximo:", lote_crudo.max(),
      "| forma:", lote_crudo.shape)
print("Normalizado    -> mínimo:", lote_norm.min(), "máximo:", lote_norm.max(),
      "| forma:", lote_norm.shape, "(la capa Flatten lo convierte en 784 dentro del modelo)")
print()
print("La etiqueta no cambia con la normalización:",
      y_train[:5].tolist(), "->", y_train[:5].tolist())

def crear_modelo_baseline(units=128, optimizer=None):
    """Perceptrón multicapa 784 -> units -> 10 sin convoluciones."""
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ])
    if optimizer is None:
        optimizer = keras.optimizers.Adam(learning_rate=0.001)
    modelo.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return modelo
```

```
tf.random.set_seed(42)
modelo_base = crear_modelo_baseline()
modelo_base.summary()

tipos = [capa.__class__.__name__ for capa in modelo_base.layers]
print()
print("Capas:", tipos)
print("¿Capas convolucionales?", any("Conv" in t for t in tipos),
      "| ¿Pooling?", any("Pooling" in t for t in tipos))
print("Parámetros entrenables:", f"{modelo_base.count_params():,}")

perdida_fn = keras.losses.SparseCategoricalCrossentropy()
optimizador = keras.optimizers.Adam(learning_rate=0.001)
BATCH_SIZE = 128
EPOCHS = 5
N_DEV = len(x_dev)


@tf.function
def paso_entrenamiento(xb, yb):
    """Un paso completo: forward -> loss -> backpropagation -> actualización."""
    with tf.GradientTape() as cinta:
        logits = modelo_base(xb, training=True)
        perdida = perdida_fn(yb, logits)
    gradientes = cinta.gradient(perdida, modelo_base.trainable_variables)
    optimizador.apply_gradients(zip(gradientes, modelo_base.trainable_variables))
    return perdida, tf.argmax(logits, axis=1)


def evaluar(x_datos, y_datos, tam_lote=1000):
    """Pérdida y exactitud por lotes, sin materializar todas las predicciones."""
    perdida_total, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_datos), tam_lote):
        xb = x_datos[inicio:inicio + tam_lote]
        yb = y_datos[inicio:inicio + tam_lote]
        xb = normalizar_lote(xb) if xb.dtype == np.uint8 else xb
        probs = modelo_base(xb, training=False).numpy()
        perdida_total += float(perdida_fn(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdida_total / total, aciertos / total


indices_train = np.arange(len(x_train))
historial = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}

for epoch in range(EPOCHS):
    np.random.shuffle(indices_train)
    suma_loss, aciertos, vistos = 0.0, 0, 0

    for inicio in range(0, len(indices_train), BATCH_SIZE):
        idx = indices_train[inicio:inicio + BATCH_SIZE]
        xb = normalizar_lote(x_train[idx])
        yb = y_train[idx]
        perdida, pred = paso_entrenamiento(xb, yb)
        suma_loss += float(perdida) * len(idx)
        aciertos += int((pred.numpy() == yb).sum())
        vistos += len(idx)

    val_loss, val_acc = evaluar(x_dev, y_dev)
    historial["loss"].append(suma_loss / vistos)
    historial["accuracy"].append(aciertos / vistos)
    historial["val_loss"].append(val_loss)
    historial["val_accuracy"].append(val_acc)
    gc.collect()
```

```
tabla_hist = pd.DataFrame({
    "epoca": range(1, EPOCHS + 1),
    "train_loss": np.round(historial["loss"], 4),
    "train_acc": np.round(historial["accuracy"], 4),
    "dev_loss": np.round(historial["val_loss"], 4),
    "dev_acc": np.round(historial["val_accuracy"], 4),
})
print("=== HISTORIAL DEL BASELINE ===")
display(tabla_hist)
print(tabla_hist.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.6))
epocas = range(1, EPOCHS + 1)

axes[0].plot(epocas, historial["loss"], marker="o", label="Train", color="#c0392b", lw=2)
axes[0].plot(epocas, historial["val_loss"], marker="s", label="Dev", color="#2980b9", lw=2,
ls="--")
axes[0].set_title("Figura 1. Baseline - Loss")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss"); axes[0].legend(fontsize=8)

axes[1].plot(epocas, historial["accuracy"], marker="o", label="Train", color="#27ae60", lw=2)
axes[1].plot(epocas, historial["val_accuracy"], marker="s", label="Dev", color="#8e44ad", lw=2,
ls="--")
axes[1].set_ylim(0.88, 1.0)
axes[1].set_title("Figura 2. Baseline - Accuracy")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy"); axes[1].legend(fontsize=8)

plt.tight_layout(); plt.savefig("figIII_01_baseline_curvas.png"); plt.show()
plt.close(fig); _ = gc.collect()

print("Referencia sin entrenar  -ln(1/10) =", round(-np.log(0.1), 4))
print("Loss Dev época 1                   =", round(historial["val_loss"][0], 4))
print("Loss Dev época final               =", round(historial["val_loss"][-1], 4))
print("Accuracy Dev época final           =", round(historial["val_accuracy"][-1], 4))
print()
print("¿El modelo supera el azar (0.10)?", historial["val_accuracy"][-1] > 0.10)
print("¿La pérdida disminuyó respecto a la referencia?",
      historial["val_loss"][-1] < -np.log(0.1))
```

Nota. El código se presenta a un espacio por tratarse de material informático, y la ejecución completa con salidas y gráficas se conserva en el repositorio (Salazar Cardenas, 2026).

Apéndice B

Código Fuente del Notebook 02: Optimización e Hiperparámetros

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import pandas as pd
import random, gc

random.seed(42)
tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)

from sklearn.model_selection import train_test_split

SEED = 42
np.random.seed(SEED)

(x_train_full, y_train_full), (x_test_raw, y_test) = keras.datasets.mnist.load_data()

# División estratificada Train / Dev (80 / 20). Test queda intocable hasta el final.
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_full,
)

# Test se normaliza completo (31 MB); Train y Dev se normalizan por lotes para
# mantener el consumo de memoria del kernel bajo control.
x_test = x_test_raw.astype("float32") / 255.0


def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n, 28, 28) en float32 (n, 28, 28) dentro de [0, 1].

    El aplanado a 784 características lo realiza la capa Flatten del modelo.
    """
    return imagenes.astype("float32") / 255.0
```

```
print("Train:", x_train.shape, "| Dev:", x_dev.shape, "| Test:", x_test.shape)


def crear_modelo_experimento(optimizador, units=128):
    """Arquitectura fija 784 -> 128 -> 10 para comparar en igualdad de condiciones."""
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ])
    modelo.compile(
        optimizer=optimizador,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return modelo


def evaluar_modelo(modelo, x_datos, y_datos, tam_lote=1000):
    perdida_fn = keras.losses.SparseCategoricalCrossentropy()
    perdida_total, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_datos), tam_lote):
        xb = x_datos[inicio:inicio + tam_lote]
        yb = y_datos[inicio:inicio + tam_lote]
        xb = normalizar_lote(xb) if xb.dtype == np.uint8 else xb
        probs = modelo(xb, training=False).numpy()
        perdida_total += float(perdida_fn(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdida_total / total, aciertos / total


def entrenar_modelo(modelo, epochs, batch_size=64, semilla=42):
    """Ciclo explícito por lotes; devuelve el historial de loss/accuracy en Train y Dev."""
    tf.random.set_seed(semilla)
    perdida_fn = keras.losses.SparseCategoricalCrossentropy()
    optimizador = modelo.optimizer

    @tf.function
    def paso(xb, yb):
        with tf.GradientTape() as cinta:
            logits = modelo(xb, training=True)
            perdida = perdida_fn(yb, logits)
        g = cinta.gradient(perdida, modelo.trainable_variables)
        optimizador.apply_gradients(zip(g, modelo.trainable_variables))
        return perdida, tf.argmax(logits, axis=1)

    indices = np.arange(len(x_train))
    hist = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}
    for _ in range(epochs):
        np.random.shuffle(indices)
        suma, aciertos, vistos = 0.0, 0, 0
        for inicio in range(0, len(indices), batch_size):
            idx = indices[inicio:inicio + batch_size]
            xb = normalizar_lote(x_train[idx])
            yb = y_train[idx]
            perdida, pred = paso(xb, yb)
            suma += float(perdida) * len(idx)
            aciertos += int((pred.numpy() == yb).sum())
            vistos += len(idx)
        val_loss, val_acc = evaluar_modelo(modelo, x_dev, y_dev)
        hist["loss"].append(suma / vistos)
        hist["accuracy"].append(aciertos / vistos)
        hist["val_loss"].append(val_loss)
```

```
        hist["val_accuracy"].append(val_acc)
        gc.collect()
    return hist


print("Funciones de experimentación definidas correctamente")

fabricas_optimizadores = {
    "SGD_momentum": lambda: keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "RMSprop": lambda: keras.optimizers.RMSprop(learning_rate=0.001),
    "Adam": lambda: keras.optimizers.Adam(learning_rate=0.001),
}

EPOCHS_OPT = 5
resultados_opt = []
historias_opt = {}

for nombre, fabrica in fabricas_optimizadores.items():
    keras.backend.clear_session()
    gc.collect()
    modelo = crear_modelo_experimento(fabrica())
    hist = entrenar_modelo(modelo, epochs=EPOCHS_OPT)
    historias_opt[nombre] = hist
    resultados_opt.append({
        "optimizador": nombre,
        "train_acc": round(hist["accuracy"][-1], 4),
        "dev_acc": round(hist["val_accuracy"][-1], 4),
        "dev_loss": round(hist["val_loss"][-1], 4),
    })
    print(f"{nombre:<14} dev_loss {hist['val_loss'][-1]:.4f} | dev_acc {hist['val_accuracy'][-1]:.
4f}")

tabla_opt = pd.DataFrame(resultados_opt)
print()
print("=== TABLA COMPARATIVA DE OPTIMIZADORES (época 5) ===")
display(tabla_opt)
print(tabla_opt.to_string(index=False))

fig, ax = plt.subplots(figsize=(7.5, 4))
colores = {"SGD_momentum": "#c0392b", "RMSprop": "#2980b9", "Adam": "#27ae60"}
for nombre, hist in historias_opt.items():
    ax.plot(range(1, EPOCHS_OPT + 1), hist["val_loss"], marker="o",
            label=nombre, color=colores[nombre], lw=2)
ax.set_title("Figura 3. Validation loss por optimizador (misma arquitectura y semilla)")
ax.set_xlabel("Época"); ax.set_ylabel("Validation loss"); ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig("figIII_02_optim_valloss.png"); plt.show()
plt.close(fig); _ = gc.collect()

learning_rates = [0.0001, 0.001, 0.01]
resultados_lr = []

for lr in learning_rates:
    keras.backend.clear_session()
    gc.collect()
    modelo = crear_modelo_experimento(keras.optimizers.Adam(learning_rate=lr))
    hist = entrenar_modelo(modelo, epochs=3)
    resultados_lr.append({
        "learning_rate": lr,
        "train_acc": round(hist["accuracy"][-1], 4),
        "dev_acc": round(hist["val_accuracy"][-1], 4),
        "dev_loss": round(hist["val_loss"][-1], 4),
    })

tabla_lr = pd.DataFrame(resultados_lr)
print("=== EFECTO DEL LEARNING RATE (Adam, 3 épocas) ===")
```

```
display(tabla_lr)
print(tabla_lr.to_string(index=False))

fig, ax = plt.subplots(figsize=(7.0, 3.8))
ax.plot([r["learning_rate"] for r in resultados_lr],
        [r["dev_acc"] for r in resultados_lr], marker="s", lw=2, color="#8e44ad")
for r in resultados_lr:
    ax.annotate(f'{r["dev_acc"]:.4f}', (r["learning_rate"], r["dev_acc"]),
                textcoords="offset points", xytext=(0, 9), ha="center", fontsize=9)
ax.set_xscale("log")
ax.set_xticks(learning_rates)
ax.set_xticklabels([str(lr) for lr in learning_rates])
ax.set_xlabel("Learning rate (Adam)")
ax.set_ylabel("Dev accuracy")
ax.set_title("Figura 4. Dev accuracy frente al learning rate")
plt.tight_layout(); plt.savefig("figIII_03_lr_dev.png"); plt.show()
plt.close(fig); _ = gc.collect()
```

Nota. El código se presenta a un espacio por tratarse de material informático, y la ejecución completa con salidas y gráficas se conserva en el repositorio (Salazar Cardenas, 2026).

Apéndice C

Código Fuente del Notebook 03: Regularización y Early Stopping

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
import pandas as pd
import random, gc

random.seed(42)
tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)

from sklearn.model_selection import train_test_split

SEED = 42
np.random.seed(SEED)

(x_train_full, y_train_full), (x_test_raw, y_test) = keras.datasets.mnist.load_data()

# División estratificada Train / Dev (80 / 20). Test queda intocable hasta el final.
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_full,
)

# Test se normaliza completo (31 MB); Train y Dev se normalizan por lotes para
# mantener el consumo de memoria del kernel bajo control.
x_test = x_test_raw.astype("float32") / 255.0


def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n, 28, 28) en float32 (n, 28, 28) dentro de [0, 1].

    El aplanado a 784 características lo realiza la capa Flatten del modelo.
    """
    return imagenes.astype("float32") / 255.0
```

```
print("Train:", x_train.shape, "| Dev:", x_dev.shape, "| Test:", x_test.shape)

# Muestra estratificada de 12 000 imágenes para el experimento de regularización
x_train_reg, _, y_train_reg, _ = train_test_split(
    x_train, y_train,
    train_size=12000,
    random_state=SEED,
    stratify=y_train,
)

print()
print("Muestra del experimento:", x_train_reg.shape, y_train_reg.shape)
print("Clases en la muestra   :", np.bincount(y_train_reg))

def entrenar_con_parada(modelo, epochs, patience=2, batch_size=64, semilla=42):
    """Entrena con ciclo explícito y Early Stopping manual sobre Dev loss."""
    tf.random.set_seed(semilla)
    perdida_fn = keras.losses.SparseCategoricalCrossentropy()
    optimizador = modelo.optimizer

    @tf.function
    def paso(xb, yb):
        with tf.GradientTape() as cinta:
            logits = modelo(xb, training=True)
            perdida = perdida_fn(yb, logits)
        g = cinta.gradient(perdida, modelo.trainable_variables)
        optimizador.apply_gradients(zip(g, modelo.trainable_variables))
        return perdida, tf.argmax(logits, axis=1)

    def evaluar_dev():
        perdida_total, aciertos, total = 0.0, 0, 0
        for inicio in range(0, len(x_dev), 1000):
            xb = normalizar_lote(x_dev[inicio:inicio + 1000])
            yb = y_dev[inicio:inicio + 1000]
            probs = modelo(xb, training=False).numpy()
            perdida_total += float(perdida_fn(yb, probs)) * len(yb)
            aciertos += int((probs.argmax(axis=1) == yb).sum())
            total += len(yb)
        return perdida_total / total, aciertos / total

    indices = np.arange(len(x_train_reg))
    hist = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}
    mejor_dev = np.inf
    mejores_pesos = None
    epocas_sin_mejora = 0
    epoch_ejecutada = 0

    for epoch in range(epochs):
        np.random.shuffle(indices)
        suma, aciertos, vistos = 0.0, 0, 0
        for inicio in range(0, len(indices), batch_size):
            idx = indices[inicio:inicio + batch_size]
            xb = normalizar_lote(x_train_reg[idx])
            yb = y_train_reg[idx]
            perdida, pred = paso(xb, yb)
            suma += float(perdida) * len(idx)
            aciertos += int((pred.numpy() == yb).sum())
            vistos += len(idx)

        dev_loss, dev_acc = evaluar_dev()
        hist["loss"].append(suma / vistos)
        hist["accuracy"].append(aciertos / vistos)
        hist["val_loss"].append(dev_loss)
        hist["val_accuracy"].append(dev_acc)
        epoch_ejecutada = epoch + 1
```

```
        if dev_loss < mejor_dev - 1e-4:
            mejor_dev = dev_loss
            mejores_pesos = [v.numpy().copy() for v in modelo.trainable_variables]
            epocas_sin_mejora = 0
        else:
            epocas_sin_mejora += 1
            if epocas_sin_mejora >= patience:
                print(f"  Early stopping en la época {epoch + 1} "
                      f"(Dev loss sin mejorar {patience} épocas seguidas)")
                break
        gc.collect()

    if mejores_pesos is not None:
        for v, peso in zip(modelo.trainable_variables, mejores_pesos):
            v.assign(peso)
        print("  Pesos restaurados a la mejor época")
    return hist, epoch_ejecutada


print("Función de entrenamiento con parada temprana definida")

def crear_modelo_sin_regularizacion():
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(512, activation="relu"),
        layers.Dense(256, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ])
    modelo.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return modelo


tf.random.set_seed(42)
modelo_sin_reg = crear_modelo_sin_regularizacion()
print("Parámetros sin regularización:", f"{modelo_sin_reg.count_params():,}")

hist_sin_reg, epocas_sin_reg = entrenar_con_parada(
    modelo_sin_reg, epochs=15, patience=100   # patience alto: deja correr las 15 épocas
)

print()
print("=== SIN REGULARIZACIÓN (15 épocas completas) ===")
print(f"{'Época':>6}{'Train loss':>12}{'Train acc':>11}{'Dev loss':>11}{'Dev acc':>10}")
for i in range(epocas_sin_reg):
    print(f"{i+1:>6}{hist_sin_reg['loss'][i]:>12.4f}{hist_sin_reg['accuracy'][i]:>11.4f}"
          f"{hist_sin_reg['val_loss'][i]:>11.4f}{hist_sin_reg['val_accuracy'][i]:>10.4f}")

def crear_modelo_regularizado():
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(512, activation="relu",
                     kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.30),
        layers.Dense(256, activation="relu",
                     kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.30),
        layers.Dense(10, activation="softmax"),
    ])
```

```
    modelo.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return modelo


tf.random.set_seed(42)
modelo_reg = crear_modelo_regularizado()
print("Parámetros regularizado:", f"{modelo_reg.count_params():,}")
print()

hist_reg, epocas_reg = entrenar_con_parada(modelo_reg, epochs=20, patience=2)

print()
print("=== REGULARIZADO (L2 + Dropout + Early Stopping) ===")
print("Épocas ejecutadas:", epocas_reg, "de 20 solicitadas")
print(f"{'Época':>6}{'Train loss':>12}{'Train acc':>11}{'Dev loss':>11}{'Dev acc':>10}")
for i in range(epocas_reg):
    print(f"{i+1:>6}{hist_reg['loss'][i]:>12.4f}{hist_reg['accuracy'][i]:>11.4f}"
          f"{hist_reg['val_loss'][i]:>11.4f}{hist_reg['val_accuracy'][i]:>10.4f}")

fig, ax = plt.subplots(figsize=(8.5, 4.4))
ax.plot(range(1, epocas_sin_reg + 1), hist_sin_reg["loss"], color="#c0392b", lw=2,
        label="Train sin regularización")
ax.plot(range(1, epocas_sin_reg + 1), hist_sin_reg["val_loss"], color="#c0392b", lw=2, ls="--",
        label="Dev sin regularización")
ax.plot(range(1, epocas_reg + 1), hist_reg["loss"], color="#27ae60", lw=2,
        label="Train regularizado")
ax.plot(range(1, epocas_reg + 1), hist_reg["val_loss"], color="#27ae60", lw=2, ls="--",
        label="Dev regularizado")
ax.set_title("Figura 5. Regularización: comparación de pérdidas Train/Dev")
ax.set_xlabel("Época"); ax.set_ylabel("Loss"); ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig("figIII_04_reg_curvas.png"); plt.show()
plt.close(fig); _ = gc.collect()

perdida_fn_eval = keras.losses.SparseCategoricalCrossentropy()


def evaluar_en_dev(modelo):
    perdida_total, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_dev), 1000):
        xb = normalizar_lote(x_dev[inicio:inicio + 1000])
        yb = y_dev[inicio:inicio + 1000]
        probs = modelo(xb, training=False).numpy()
        perdida_total += float(perdida_fn_eval(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdida_total / total, aciertos / total


def evaluar_en_muestra(modelo):
    """Mismas métricas sobre la muestra de entrenamiento (pesos ya restaurados)."""
    perdida_total, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_train_reg), 1000):
        xb = normalizar_lote(x_train_reg[inicio:inicio + 1000])
        yb = y_train_reg[inicio:inicio + 1000]
        probs = modelo(xb, training=False).numpy()
        perdida_total += float(perdida_fn_eval(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdida_total / total, aciertos / total
```

```
# Todas las métricas se calculan con los pesos restaurados a la mejor época,
# de modo que Train y Dev sean comparables entre sí.
dev_sin = evaluar_en_dev(modelo_sin_reg)
dev_con = evaluar_en_dev(modelo_reg)
train_sin = evaluar_en_muestra(modelo_sin_reg)
train_con = evaluar_en_muestra(modelo_reg)

brecha_sin = train_sin[1] - dev_sin[1]
brecha_con = train_con[1] - dev_con[1]

tabla_reg = pd.DataFrame([
    {"modelo": "Sin regularización", "epocas": epocas_sin_reg,
     "train_acc": round(train_sin[1], 4),
     "dev_loss": round(dev_sin[0], 4), "dev_acc": round(dev_sin[1], 4),
     "brecha_train_dev": round(brecha_sin, 4)},
    {"modelo": "L2 + Dropout + Early Stopping", "epocas": epocas_reg,
     "train_acc": round(train_con[1], 4),
     "dev_loss": round(dev_con[0], 4), "dev_acc": round(dev_con[1], 4),
     "brecha_train_dev": round(brecha_con, 4)},
])
print("=== COMPARACIÓN EN DEV ===")
display(tabla_reg)
print(tabla_reg.to_string(index=False))
print()
print("La brecha Train-Dev mide el overfitting: cuanto menor, mejor generaliza el modelo.")
print("Métricas calculadas con los pesos restaurados a la mejor época (restore_best_weights).")
```

Nota. El código se presenta a un espacio por tratarse de material informático, y la ejecución completa con salidas y gráficas se conserva en el repositorio (Salazar Cardenas, 2026).

Apéndice D

Código Fuente del Notebook 04: Métricas de Evaluación sobre Test

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay,
                             classification_report)
import pandas as pd
import random, gc

random.seed(42)
tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)

from sklearn.model_selection import train_test_split

SEED = 42
np.random.seed(SEED)

(x_train_full, y_train_full), (x_test_raw, y_test) = keras.datasets.mnist.load_data()

# División estratificada Train / Dev (80 / 20). Test queda intocable hasta el final.
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_full,
)

# Test se normaliza completo (31 MB); Train y Dev se normalizan por lotes para
# mantener el consumo de memoria del kernel bajo control.
x_test = x_test_raw.astype("float32") / 255.0


def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n, 28, 28) en float32 (n, 28, 28) dentro de [0, 1].

    El aplanado a 784 características lo realiza la capa Flatten del modelo.
    """
```

```
    return imagenes.astype("float32") / 255.0


print("Train:", x_train.shape, "| Dev:", x_dev.shape, "| Test:", x_test.shape)

x_train_reg, _, y_train_reg, _ = train_test_split(
    x_train, y_train, train_size=12000, random_state=SEED, stratify=y_train
)
print()
print("Muestra de entrenamiento del modelo final:", x_train_reg.shape)

tf.random.set_seed(42)
modelo_reg = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(512, activation="relu", kernel_regularizer=regularizers.l2(1e-4)),
    layers.Dropout(0.30),
    layers.Dense(256, activation="relu", kernel_regularizer=regularizers.l2(1e-4)),
    layers.Dropout(0.30),
    layers.Dense(10, activation="softmax"),
])
modelo_reg.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

perdida_fn = keras.losses.SparseCategoricalCrossentropy()
optimizador = modelo_reg.optimizer


@tf.function
def paso(xb, yb):
    with tf.GradientTape() as cinta:
        logits = modelo_reg(xb, training=True)
        perdida = perdida_fn(yb, logits)
    g = cinta.gradient(perdida, modelo_reg.trainable_variables)
    optimizador.apply_gradients(zip(g, modelo_reg.trainable_variables))
    return perdida, tf.argmax(logits, axis=1)


def dev_loss_acc():
    pt, ac, tot = 0.0, 0, 0
    for inicio in range(0, len(x_dev), 1000):
        xb = normalizar_lote(x_dev[inicio:inicio + 1000])
        yb = y_dev[inicio:inicio + 1000]
        probs = modelo_reg(xb, training=False).numpy()
        pt += float(perdida_fn(yb, probs)) * len(yb)
        ac += int((probs.argmax(axis=1) == yb).sum())
        tot += len(yb)
    return pt / tot, ac / tot


indices = np.arange(len(x_train_reg))
mejor_dev, mejores_pesos, sin_mejora, epocas_reg = np.inf, None, 0, 0
for epoch in range(20):
    np.random.shuffle(indices)
    for inicio in range(0, len(indices), 64):
        idx = indices[inicio:inicio + 64]
        paso(normalizar_lote(x_train_reg[idx]), y_train_reg[idx])
    dl, _ = dev_loss_acc()
    epocas_reg = epoch + 1
    if dl < mejor_dev - 1e-4:
        mejor_dev, sin_mejora = dl, 0
        mejores_pesos = [v.numpy().copy() for v in modelo_reg.trainable_variables]
```

```
    else:
        sin_mejora += 1
        if sin_mejora >= 2:
            break
    gc.collect()

for v, peso in zip(modelo_reg.trainable_variables, mejores_pesos):
    v.assign(peso)
print("Épocas ejecutadas:", epocas_reg, "| pesos restaurados a la mejor época (Dev loss =",
      round(mejor_dev, 4), ")")

probabilidades_test = []
for inicio in range(0, len(x_test), 1000):
    lote = modelo_reg.predict(x_test[inicio:inicio + 1000], verbose=0)
    probabilidades_test.append(lote)
probabilidades_test = np.vstack(probabilidades_test)
y_pred = np.argmax(probabilidades_test, axis=1)

print("Predicciones sobre Test:", y_pred.shape)
print("Primeras 20 predicciones:", y_pred[:20].tolist())
print("Primeras 20 clases reales:", y_test[:20].tolist())
print("Coincidencias en la muestra:", int((y_pred[:20] == y_test[:20]).sum()), "de 20")

accuracy = accuracy_score(y_test, y_pred)
precision_macro = precision_score(y_test, y_pred, average="macro", zero_division=0)
recall_macro = recall_score(y_test, y_pred, average="macro", zero_division=0)
f1_macro = f1_score(y_test, y_pred, average="macro", zero_division=0)

metricas = pd.DataFrame({
    "metrica": ["Accuracy", "Precision macro", "Recall macro", "F1 macro"],
    "resultado": [round(accuracy, 4), round(precision_macro, 4),
                  round(recall_macro, 4), round(f1_macro, 4)],
})
print("=== MÉTRICAS FINALES SOBRE TEST ===")
display(metricas)
print(metricas.to_string(index=False))
print()
print("Accuracy   :", round(accuracy, 4))
print("Precision  :", round(precision_macro, 4))
print("Recall     :", round(recall_macro, 4))
print("F1 macro   :", round(f1_macro, 4))

reporte = classification_report(y_test, y_pred, digits=4, zero_division=0)
print(reporte)

lineas = [l.split() for l in reporte.splitlines()
          if l.strip() and l.strip()[0].isdigit()]
peor_f1, peor_digito = 1.0, None
for partes in lineas:
    f1_clase = float(partes[3])
    if f1_clase < peor_f1:
        peor_f1, peor_digito = f1_clase, partes[0]
print()
print(f"Clase con menor F1: dígito {peor_digito} (F1 = {peor_f1:.4f}) "
      f"frente al macro {f1_macro:.4f}")

cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(6.0, 5.0))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.arange(10))
disp.plot(ax=ax, cmap="Blues", colorbar=True, values_format="d", xticks_rotation=0)
ax.set_title("Figura 6. Matriz de confusión - MNIST (Test, 10 000 imágenes)")
plt.tight_layout(); plt.savefig("figIII_05_confusion.png"); plt.show()
plt.close(fig); _ = gc.collect()
```

```
print("Total de errores en Test:", int((y_pred != y_test).sum()), "de", len(y_test))

cm_errores = cm.copy()
np.fill_diagonal(cm_errores, 0)

real_confundido, predicho_como = np.unravel_index(np.argmax(cm_errores), cm_errores.shape)
cantidad = int(cm_errores[real_confundido, predicho_como])

print(f"La confusión más frecuente fue: {real_confundido} clasificado como "
      f"{predicho_como} ({cantidad} veces)")

pares = sorted([(cm_errores[r, c], r, c) for r in range(10) for c in range(10)], reverse=True)
print()
print("Top 5 confusiones (real -> predicho, casos):")
for cant, r, c in pares[:5]:
    print(f"   {r} -> {c} : {int(cant)}")

indices_error = np.where((y_test == real_confundido) & (y_pred == predicho_como))[0]

fig, axes = plt.subplots(1, min(6, len(indices_error)), figsize=(9.5, 2.4))
axes = np.atleast_1d(axes)
for ax, idx in zip(axes, indices_error[:6]):
    ax.imshow(x_test_raw[idx], cmap="gray")
    ax.set_title(f"Real {y_test[idx]}\nPred {y_pred[idx]}", fontsize=8, color="#c0392b")
    ax.axis("off")
fig.suptitle(f"Figura 7. Ejemplos reales de la confusión {real_confundido} -> {predicho_como}",
             y=1.08)
plt.tight_layout(); plt.savefig("figIII_06_error_muestra.png"); plt.show()
plt.close(fig); _ = gc.collect()

idx0 = int(indices_error[0])
print("Imagen analizada: índice", idx0,
      "| confianza del modelo:", round(float(probabilidades_test[idx0].max()), 4))
```

Nota. El código se presenta a un espacio por tratarse de material informático, y la ejecución completa con salidas y gráficas se conserva en el repositorio (Salazar Cardenas, 2026).

Apéndice E

Código Fuente del Notebook 05: Réplica con PyTorch

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
import pandas as pd
import urllib.request
import os
import gc

torch.manual_seed(42)
print("PyTorch:", torch.__version__)
print("Dispositivo:", "cuda" if torch.cuda.is_available() else "cpu")

SEED = 42

RUTA_NPZ = "mnist.npz"
if not os.path.exists(RUTA_NPZ):
    print("Descargando mnist.npz (~11 MB)...")
    urllib.request.urlretrieve(
        "https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz",
        RUTA_NPZ,
    )

with np.load(RUTA_NPZ) as datos:
    x_train_full, y_train_full = datos["x_train"], datos["y_train"]
    x_test_raw, y_test_np = datos["x_test"], datos["y_test"]

print("Train completo:", x_train_full.shape, "| Test:", x_test_raw.shape)

x_train_np, x_dev_np, y_train_np, y_dev_np = train_test_split(
    x_train_full, y_train_full, test_size=0.20, random_state=SEED, stratify=y_train_full
)

# Train como tensor uint8 (47 MB); la normalización a [0,1] ocurre por lote.
train_dataset = TensorDataset(
    torch.tensor(x_train_np, dtype=torch.uint8),
    torch.tensor(y_train_np, dtype=torch.long),
)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

del x_train_full, y_train_full
_ = gc.collect()
```

```
print("Train:", len(train_dataset), "| Dev:", len(x_dev_np), "| Test:", len(x_test_raw))

class RedMNISTRegularizada(nn.Module):
    """784 -> 256 -> 10 con Dropout(0.30); L2 entra por weight_decay del optimizador."""

    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.30)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        return self.fc2(x)


modelo_torch = RedMNISTRegularizada()
criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_torch.parameters(), lr=0.001, weight_decay=1e-4)

print(modelo_torch)
total = sum(p.numel() for p in modelo_torch.parameters())
print()
print("Parámetros entrenables:", f"{total:,}")

def evaluar_conjunto(x_np, y_np, tam_lote=1000):
    """Pérdida y exactitud sobre un conjunto NumPy uint8, por bloques."""
    modelo_torch.eval()
    suma_loss, aciertos, total = 0.0, 0, 0
    with torch.no_grad():
        for inicio in range(0, len(x_np), tam_lote):
            xb = torch.tensor(
                x_np[inicio:inicio + tam_lote].reshape(-1, 784) / 255.0,
                dtype=torch.float32,
            )
            yb = torch.tensor(y_np[inicio:inicio + tam_lote], dtype=torch.long)
            out = modelo_torch(xb)
            suma_loss += criterio(out, yb).item() * len(yb)
            aciertos += int((out.argmax(1) == yb).sum())
            total += len(yb)
    return suma_loss / total, aciertos / total, aciertos


EPOCHS = 3
historial = {"train_loss": [], "train_acc": [], "dev_loss": [], "dev_acc": []}

for epoch in range(EPOCHS):
    modelo_torch.train()
    suma_loss, aciertos, total_obs = 0.0, 0, 0

    for imagenes, etiquetas in train_loader:
        xb = imagenes.float().reshape(len(imagenes), -1) / 255.0  # normaliza el lote

        optimizador.zero_grad()              # 1. limpiar gradientes acumulados
        salidas = modelo_torch(xb)           # 2. forward propagation
        error = criterio(salidas, etiquetas)  # 3. loss
        error.backward()                     # 4. backpropagation
        optimizador.step()                   # 5. actualización de pesos

        suma_loss += error.item() * len(etiquetas)
```

```
        aciertos += int((salidas.argmax(1) == etiquetas).sum())
        total_obs += len(etiquetas)

    dev_loss, dev_acc, _ = evaluar_conjunto(x_dev_np, y_dev_np)
    historial["train_loss"].append(suma_loss / total_obs)
    historial["train_acc"].append(aciertos / total_obs)
    historial["dev_loss"].append(dev_loss)
    historial["dev_acc"].append(dev_acc)
    print(f"Época {epoch+1} | train_loss {suma_loss/total_obs:.4f} "
          f"train_acc {aciertos/total_obs:.4f} | dev_loss {dev_loss:.4f} "
          f"dev_acc {dev_acc:.4f}")
    gc.collect()

tabla_torch = pd.DataFrame({
    "epoca": range(1, EPOCHS + 1),
    "train_loss": np.round(historial["train_loss"], 4),
    "train_acc": np.round(historial["train_acc"], 4),
    "dev_loss": np.round(historial["dev_loss"], 4),
    "dev_acc": np.round(historial["dev_acc"], 4),
})
print()
display(tabla_torch)

fig, ax = plt.subplots(figsize=(7.0, 3.8))
ax.plot(range(1, EPOCHS + 1), historial["train_loss"], marker="o", lw=2, label="Train loss")
ax.plot(range(1, EPOCHS + 1), historial["dev_loss"], marker="s", lw=2, ls="--", label="Dev loss")
ax.set_title("Figura 8. PyTorch - loss de entrenamiento y validación")
ax.set_xlabel("Época"); ax.set_ylabel("Loss"); ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig("figIII_07_pytorch_curvas.png"); plt.show()
plt.close(fig); _ = gc.collect()

test_loss, test_acc, correctas = evaluar_conjunto(x_test_raw, y_test_np)

print("=== RESULTADO FINAL · PyTorch ===")
print(f"Accuracy en Test: {test_acc:.4f} ({int(correctas)}/{len(y_test_np)})")
print(f"Loss en Test    : {test_loss:.4f}")
```

Nota. El código se presenta a un espacio por tratarse de material informático, y la ejecución completa con salidas y gráficas se conserva en el repositorio (Salazar Cardenas, 2026).